Experiment No. 10

Distance Vector Routing Algorithm

Name: Parth Vishnu PRN: 24070521279

Simulation of the Distance Vector Routing Algorithm to compute shortest paths using C

Objective:

- To understand how routers share distance vectors to compute the shortest paths.
- To implement the Distance Vector Routing algorithm in C.
- To generate the routing table (next hop and distance) for every node.

Software/Tools Required:

- Linux (Ubuntu) system
- GCC compiler
- Text editor (vim / nano) and Terminal

Step-by-step Procedure:

Algorithm: Distance Vector Routing

Input: number of nodes N, and cost matrix C[N][N] where C[i][j] is the cost from node i to node j.

- For each node i, initialize its distance vector DV[i][j] = C[i][j] for all j. Set the next hop to j if there is a direct connection, otherwise to -1.
- Repeat until no distance vector changes:
  - For each node i, each destination j and each neighbor k of node i:
  - If DV[i][j] > C[i][k] + DV[k][j], update DV[i][j] = C[i][k] + DV[k][j] and set next hop to k.
- After convergence, print the final distance vectors (routing tables) of all nodes.

Compile and Run

Step 1: Write the program

```
vim exp10_279.c
```

Step 2: Compile the program

```
gcc exp10_279.c -o exp10_279
```

Step 3: Run the program

```
./exp10_279
```

Step 4: Provide input

- Enter the number of nodes (3) and the cost matrix row by row (use 999 for no connection).

Observation Table

Step	Command / Action	Observation
1	Input: 3 nodes; cost matrix 0 2 7 / 2 0 1 / 7 1 0	Program accepted the cost matrix
2	Routing table for Node 0	To 1: next hop 1, distance 2; to 2: next hop 1, distance 3
3	Routing table for Node 1	To 0: next hop 0, distance 2; to 2: next hop 2, distance 1
4	Routing table for Node 2	To 0: next hop 1, distance 3; to 1: next hop 1, distance 1

Result:

- Successfully implemented the Distance Vector Routing algorithm in C.
- The shortest path from Node 0 to Node 2 is 3 (via Node 1), better than the direct cost of 7.

Conclusion:

- Each node maintains a table with the distance to every other node and the next hop to reach it.
- The algorithm loops until no updates are made, which ensures the shortest paths are computed.

Output / Screenshots

```c
#include <stdio.h>
#define MAX_NODES 10
#define INF 9999
struct Node {
    int dist[MAX_NODES];
    int from[MAX_NODES];
} rt[MAX_NODES];

int main() {
    int cost[MAX_NODES][MAX_NODES], nodes, i, j, k;
    int updated, iteration;
    printf("Enter number of nodes: ");
    scanf("%d", &nodes);
    printf("Enter cost matrix row by row (use 999 for INF/no connection):\n");
    for (i = 0; i < nodes; i++) {
        for (j = 0; j < nodes; j++) {
            int v;
            scanf("%d", &v);
            if (v == 999) cost[i][j] = INF;
            else cost[i][j] = v;
        }
    }
    for (i = 0; i < nodes; i++) {
        cost[i][i] = 0;
        for (j = 0; j < nodes; j++) {
            rt[i].dist[j] = cost[i][j];
            if (cost[i][j] < INF && i != j) rt[i].from[j] = j; // next hop is the neighbor j
            else rt[i].from[j] = -1; // unknown/unreachable
        }
    }
    /* Repeat relaxations at most nodes-1 times (Bellman-Ford style). */
    for (iteration = 0; iteration < nodes - 1; iteration++) {
        updated = 0;
        for (i = 0; i < nodes; i++) {
            for (j = 0; j < nodes; j++) {
                for (k = 0; k < nodes; k++) {
                    if (cost[i][k] >= INF) continue;         // no direct link i->k
                    if (rt[k].dist[j] >= INF) continue;      // k->j not known
                    int newdist = cost[i][k] + rt[k].dist[j];
                    if (newdist < rt[i].dist[j]) {
                        rt[i].dist[j] = newdist;
                        rt[i].from[j] = k;
                        updated = 1;
                    }
                }
            }
        }
        if (!updated) break;
    }

    // Display routing table
    for (i = 0; i < nodes; i++) {
        printf("\nRouting table for Node %d:\n", i);
        printf("Destination\tNext Hop\tDistance\n");
        for (j = 0; j < nodes; j++) {
            if (rt[i].dist[j] >= INF)
                printf("%d\t\t%s\t\t%s\n", j, (rt[i].from[j] == -1 ? "-" : "-"), "INF");
            else
                printf("%d\t\t%d\t\t%d\n", j, rt[i].from[j], rt[i].dist[j]);
        }
    }

    return 0;
}
```

Fig 1: Code

```
sit-lab2-pc29@sit-lab2-pc29-OptiPlex-3280-AIO:~$ vim exp10_279.c
sit-lab2-pc29@sit-lab2-pc29-OptiPlex-3280-AIO:~$ gcc exp10_279.c -o exp10_279
sit-lab2-pc29@sit-lab2-pc29-OptiPlex-3280-AIO:~$ ./exp10_279
Enter number of nodes: 3
Enter cost matrix row by row (use 999 for INF/no connection):
0 2 7
2 0 1
7 1 0

Routing table for Node 0:
Destination     Next Hop        Distance
0               -1              0
1               1               2
2               1               3

Routing table for Node 1:
Destination     Next Hop        Distance
0               0               2
1               -1              0
2               2               1

Routing table for Node 2:
Destination     Next Hop        Distance
0               1               3
1               1               1
2               -1              0
sit-lab2-pc29@sit-lab2-pc29-OptiPlex-3280-AIO:~$
```

Fig 2: Output - routing tables